

What We'll Cover Today

- 1 What is Node.js?
- 2 Why do we need Node.js?
- 3 How Node.js executes code
- 4 Event-driven architecture
- 5 Non-blocking I/O
- 6 V8 Engine
- 7 Event Loop
- 8 Node.js Modules
- 9 npm basics
- 10 First Node.js application

KEY QUESTION

How can one Node.js application handle thousands of requests without creating one thread for every request?

What is Node.js?

- A JavaScript runtime environment
- Built on Google's V8 JavaScript engine
- Executes JavaScript outside the browser
- Designed for server-side applications
- Particularly effective for I/O-heavy apps

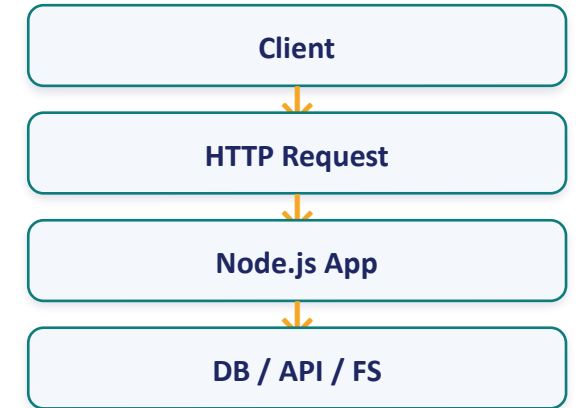
Important: *Node.js is NOT a programming language.*

JavaScript = Language **Node.js** = Runtime

Traditional JavaScript

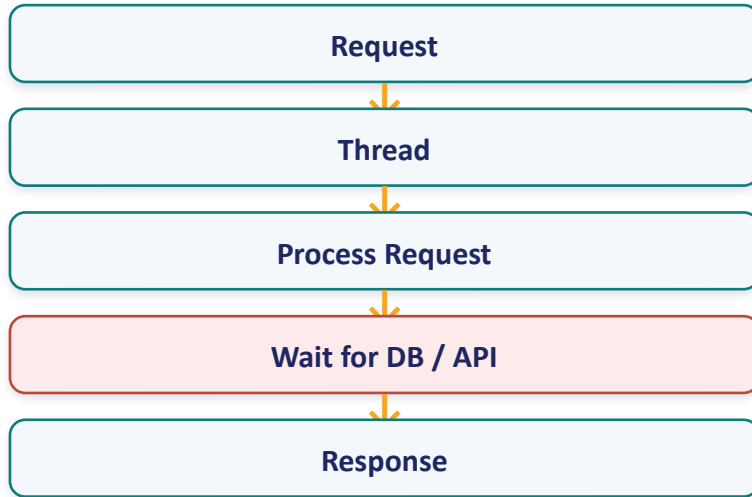


With Node.js

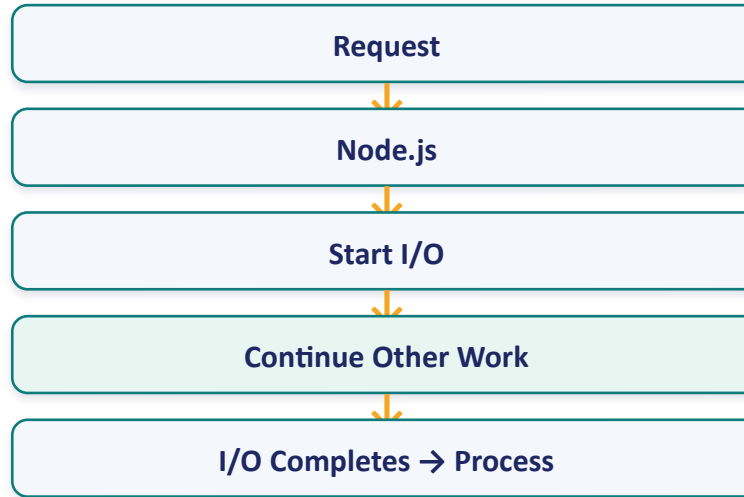


Why Node.js?

Traditional approach



Node.js approach



A lot of time can be spent waiting

Best suited for

REST APIs

Microservices

Real-time apps

Chat applications

Streaming apps

API gateways

BFF layers

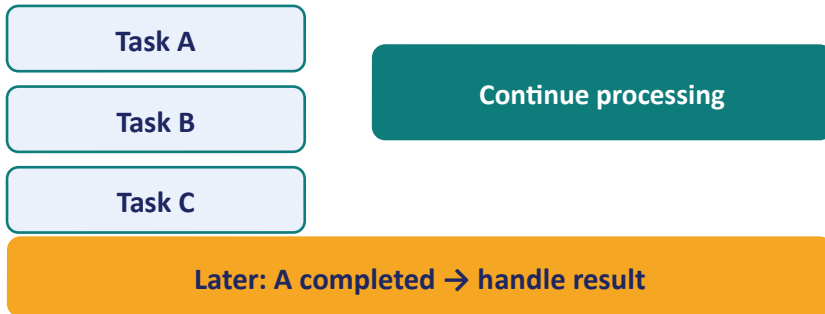
Synchronous vs Asynchronous

Synchronous



Task B cannot start until Task A finishes.

Asynchronous



Real-world example — ordering food

Synchronous: Order → stand at counter → wait → receive food → leave

Asynchronous: Order → get token → sit down → do something else → food ready → collect

Async does not mean instant.

It means we don't have to stop everything while waiting for the operation to finish.

Note: Asynchronous ≠ Multithreaded

What is I/O?

I/O = Input / Output

Reading a file

Writing a file

Database query

Calling a REST API

Reading from network

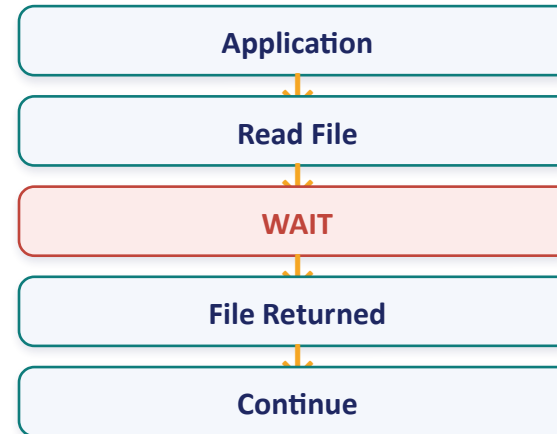
Sending a network response

app.js

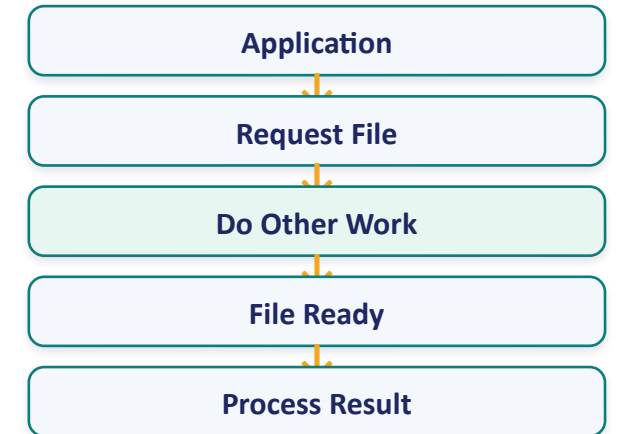
```
const data = readFile();  
console.log(data);
```

```
// app may need to wait for the file system
```

Blocking model



Non-blocking model



Event-Driven Architecture

Instead of constantly asking "is it finished?", Node.js responds when an event occurs.



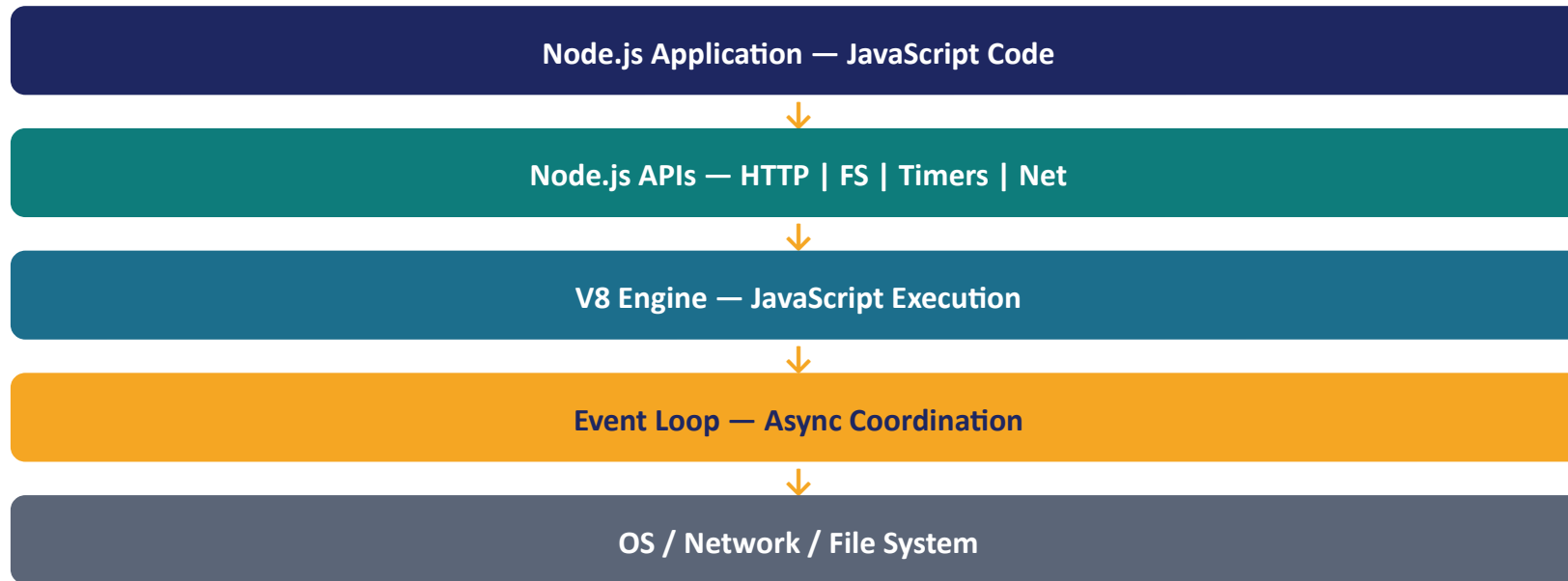
Common events

- Request received
- File read completed
- Database query completed
- Timer expired
- Network response received

Event → Handler

Node.js is very good at coordinating lots of events and I/O operations at once.

Node.js Architecture



V8

Runs JavaScript

Node.js APIs

Access to FS, networking, HTTP, timers, processes

Event Loop

Coordinates asynchronous work

libuv

Underlying async I/O infrastructure

V8 Engine

V8 is Google's open-source JavaScript engine — used by Chrome and Node.js.



app.js

```
const name = "Rahul";  
  
console.log(`Hello ${name}`);
```

Chrome runs JavaScript because it contains V8. Node.js runs JavaScript because it also uses V8.

JavaScript

Programming Language

V8

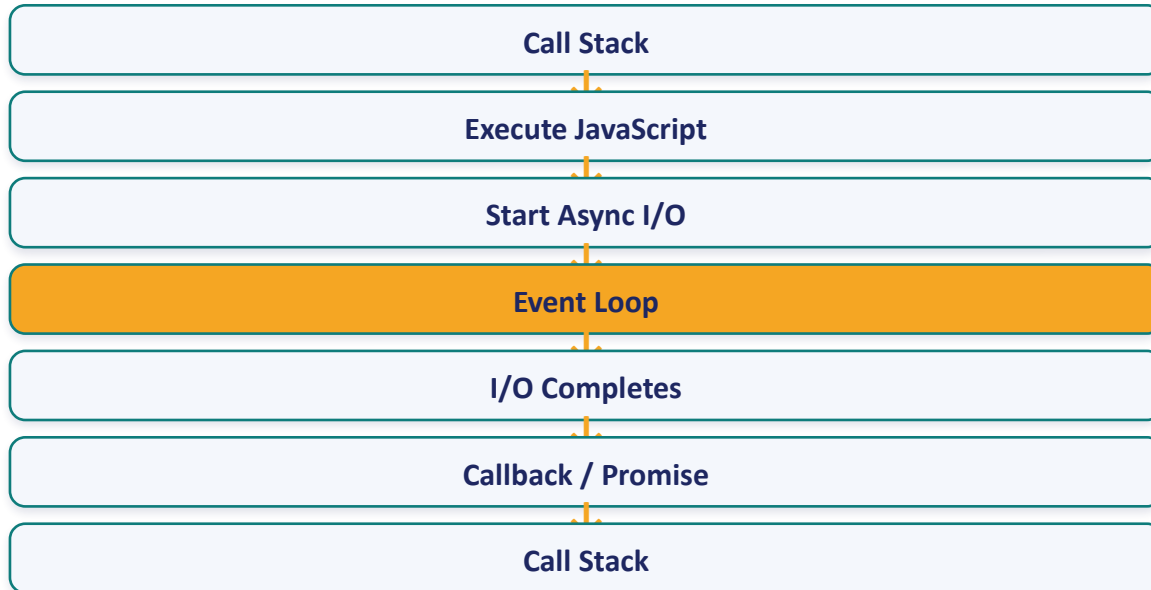
JavaScript Engine

Node.js

Runtime Environment

The Node.js Event Loop

Node.js uses an event loop to coordinate asynchronous operations.



demo2.js

```
console.log("Start");

setTimeout(() => {
  console.log("Timer finished");
}, 2000);

console.log("End");
```

Expected output

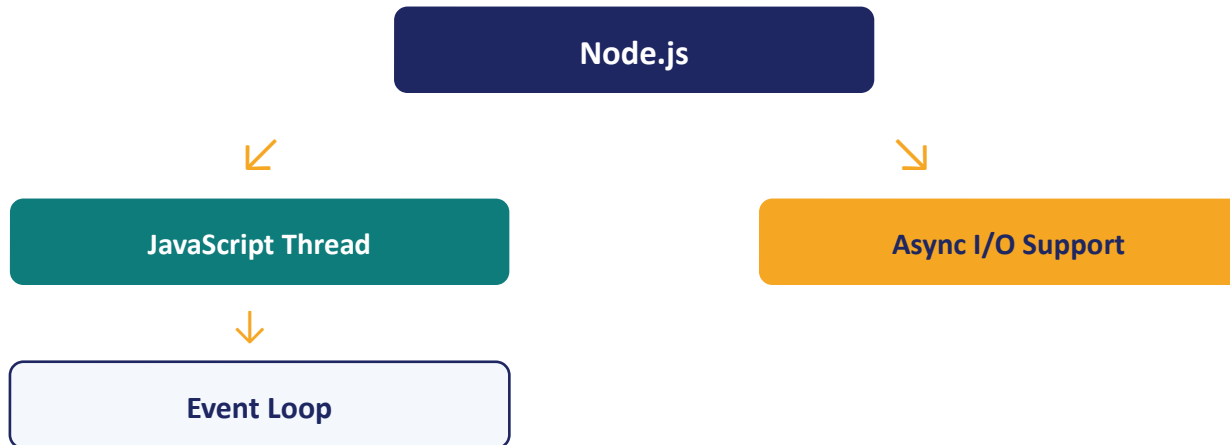
OUTPUT

```
Start
End
Timer finished
```

The event loop does not make JS run in parallel — it coordinates asynchronous operations.

Is Node.js Single-Threaded?

Short answer: *JavaScript execution is primarily single-threaded.*



Single-threaded does not mean incapable of concurrency. Node.js can handle many concurrent I/O operations.

What this means

One main JavaScript thread handles:

- JavaScript execution
- Event handling
- Callbacks

Node.js also uses underlying system mechanisms and worker threads for certain operations.

Analogy

One receptionist can coordinate many customers — as long as they don't personally perform every waiting operation.

Node.js + npm

npm = Node Package Manager — install packages, manage dependencies, run scripts, publish packages.

shell

```
$ node --version
$ npm --version

$ mkdir node-demo && cd node-demo
$ npm init -y
# creates package.json
```

package.json

```
{
  "name": "node-demo",
  "version": "1.0.0",
  "main": "app.js"
}
```

Java / Maven → Node.js / npm

pom.xml



package.json

package.json is the project metadata and dependency-definition file — the Node.js equivalent of pom.xml.

This is where students switch from theory to hands-on.

Run `node --version` and `npm --version`, then create a project with `npm init -y` and inspect the generated `package.json` together.

First Node.js Application

app.js

```
console.log("Hello Node.js!");
```

shell

```
$ node app.js  
Hello Node.js!
```

app.js (v2)

```
const name = "Student";  
  
console.log(`Hello ${name}`);  
console.log("Welcome to Node.js");
```

No main(), class, public, or static required.

Hello.java

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

app.js

```
console.log("Hello");
```

JavaScript has a much lighter syntax for getting started — but real Node.js apps still have structure, modules, packages, config & frameworks.

What is a Module?

A module is a reusable piece of code. Instead of one giant app.js, we split the application.

project/

├─ app.js

├─ calculator.js

└─ user.js

Why modules?

Reusability

Maintainability

Separation of concerns

Easier testing

Cleaner code

calculator.js

```
function add(a, b) {  
  return a + b;  
}  
  
module.exports = { add };
```

app.js

```
const calculator = require("./calculator");  
  
console.log(calculator.add(10, 20));  
// → 30
```

Java connection

Java doesn't put the whole app in one class — we create classes and packages. Node.js follows the same idea: break large apps into reusable modules.

Core Modules

Node.js ships with built-in modules — no installation required.

fs

File system

http

HTTP server

path

File paths

os

OS information

events

Event handling

crypto

Cryptographic functions

x npm install os — NOT required

os is a Node.js built-in (core) module.

app.js

```
const os = require("os");

console.log(os.platform());
console.log(os.arch());
console.log(os.cpus().length);
```

Three module categories

Core require("fs") — comes with Node.js

Third-party require("express") — via npm install

Custom require("./calculator") — created by us

Module Types

Modules

Core

fs

http

os

Third-party

express

lodash

axios

Custom

calculator

user

database

loading modules

```
// Core
const fs = require("fs");

// Third-party
const express = require("express");
```

custom

```
// Custom (note the ./)
const calculator =
require("./calculator");
```

Which needs npm install?

fs → No

express → Yes

calculator → No

Hour 1 Summary

Node.js

JavaScript runtime for server-side apps

Event Loop

Coordinates asynchronous operations

Single-threaded JS

Main execution occurs on one primary thread

Modules

Organize and reuse code

V8

Executes JavaScript

Non-blocking I/O

App continues while I/O is in progress

npm

Package and project management

Mental model

JavaScript



V8



Node.js



Event Loop + APIs



Network / DB / Files

Five closing questions

Is Node.js a language? • What does V8 do? • Why is non-blocking I/O useful? • What is the Event Loop? • fs vs express vs ./calculator?

Suggested 60-Minute Delivery

TIME	SLIDES / ACTIVITY	FOCUS
0–5 min	Slide 1	Introduction
5–10 min	Slide 2	What is Node.js
10–15 min	Slide 3	Why Node.js
15–22 min	Slides 4–5	Async + Non-blocking I/O
22–28 min	Slide 6	Event-driven architecture
28–38 min	Slides 7–10	Architecture, V8, Event Loop
38–45 min	Live Demo	Event Loop example
45–50 min	Slides 11–12	npm + Hello World
50–57 min	Slides 13–15	Modules
57–60 min	Slide 16	Recap + questions

Live Demo Progression

Don't make this hour purely PPT-based — run these three demos progressively.

1

Hello World

```
demo 1
$ node app.js
Hello World
```

2

Event Loop

```
demo 2
console.log("1");
setTimeout(() =>
  console.log("2"), 2000);
console.log("3");
// → 1, 3, 2
```

3

Custom Module

```
demo 3
calculator.js
  → module.exports
app.js
  → require("./calculator")
```

Run JavaScript → Understand Node execution → Build reusable Node code

Sets up the next stages naturally: HTTP server → Express → REST API → JSON → database → microservices → Docker → CI/CD — mapping to the Java/Spring Boot concepts learned in parallel.